Bloom Filters

with Discrete Probability Review

Hoa T. Vu

Discrete Probability Review

Probability Space

A **probability space** is a triple $(\Omega, \mathcal{F}, \Pr)$:

- Ω: **Sample space** — the set of all possible outcomes.
- $\mathcal{F} \subseteq 2^{\Omega}$: **Event space**. An event is a subset of the outcomes.
- $\Pr : \mathcal{F} \to [0, 1]$: **Probability measure** — assigns a probability to each event.

Probability Space

A **probability space** is a triple $(\Omega, \mathcal{F}, \Pr)$:

- Ω: **Sample space** — the set of all possible outcomes.
- $\mathcal{F} \subseteq 2^{\Omega}$: **Event space**. An event is a subset of the outcomes.
- $\Pr : \mathcal{F} \to [0, 1]$: **Probability measure** — assigns a probability to each event.

Axioms:

1. $\Pr(\Omega) = 1$.
2. $\Pr(A) \geq 0$ for all $A \in \mathcal{F}$.
3. If $A_1, A_2, \ldots$ are pairwise disjoint, then $\Pr(\bigcup_i A_i) = \sum_i \Pr(A_i)$.

Probability Space — Example

Example: Roll a fair six-sided die.

- $\Omega = \{1, 2, 3, 4, 5, 6\}$
- $\Pr(\{\omega\}) = 1/6$ for each $\omega \in \Omega$
- The event "roll an odd number" would be $\{1, 3, 5\}$

Probability Space — Example

Example: Flip two fair coins.

- $\Omega = \{HH, HT, TH, TT\}$
- $\Pr(\{\omega\}) = 1/4$ for each $\omega \in \Omega$
- The event "two different outcomes" would be $\{HT, TH\}$

Events

An **event** is a subset $A \subseteq \Omega$.

- A **occurs** if the outcome $\omega \in A$.
- **Complement:** $\bar{A} = \Omega \setminus A, \quad \Pr(\bar{A}) = 1 - \Pr(A)$.
- **Union:** $\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$.
- **Union bound:** $\Pr(A \cup B) \leq \Pr(A) + \Pr(B)$.

Events — Example

Roll a die. Let $A = \{2, 4, 6\}$ (even), $B = \{1, 2, 3\}$ (at most 3).

- $\Pr(A) = 3/6 = 1/2$
- $\Pr(B) = 3/6 = 1/2$
- $A \cap B = \{2\}$, so $\Pr(A \cap B) = 1/6$
- $\Pr(A \cup B) = 1/2 + 1/2 - 1/6 = 5/6$

Independence

Events A and B are **independent** if

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B).$$

Example: Flip two fair coins. Let $A =$ "first coin is H", $B =$ "second coin is H".

- $\Pr(A) = 1/2, \quad \Pr(B) = 1/2$
- $A \cap B = \{HH\}, \quad \Pr(A \cap B) = 1/4 = \Pr(A) \cdot \Pr(B)$

Independence

Events A and B are **independent** if

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B).$$

Example: Flip two fair coins. Let $A =$ "first coin is H", $B =$ "second coin is H".

- $\Pr(A) = 1/2, \quad \Pr(B) = 1/2$
- $A \cap B = \{HH\}, \quad \Pr(A \cap B) = 1/4 = \Pr(A) \cdot \Pr(B)$

Conditional probability:

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}, \quad \text{provided } \Pr(B) > 0.$$

If A and B are independent, then $\Pr(A \mid B) = \Pr(A)$.

Random Variables

A **random variable** X is a function $X : \Omega \to \mathbb{R}$.

- Assigns a numerical value to each outcome.
- The **range** of X is the set of values X can take.

Random Variables

A **random variable** X is a function $X : \Omega \to \mathbb{R}$.

- Assigns a numerical value to each outcome.
- The **range** of X is the set of values X can take.

Example: Roll two dice. Let $X =$ sum of the two dice.

- $\Omega = \{(i, j) : 1 \leq i, j \leq 6\}, \quad |\Omega| = 36$
- $X((i, j)) = i + j$
- Range of X: $\{2, 3, \ldots, 12\}$
- $\Pr(X = 7) = 6/36 = 1/6$

Probability Mass Function

For a discrete random variable X, the **probability mass function (PMF)** is

$$p_X(x) = \Pr(X = x).$$

Probability Mass Function

For a discrete random variable X, the **probability mass function (PMF)** is

$$p_X(x) = \Pr(X = x).$$

Properties:

- $p_X(x) \geq 0$ for all x.
- $\sum_x p_X(x) = 1$.

Expectation

The **expectation** (or expected value) of a discrete random variable X:

$$\mathbb{E}[X] = \sum_x x \cdot \Pr(X = x).$$

Expectation

The **expectation** (or expected value) of a discrete random variable X:

$$\mathbb{E}[X] = \sum_x x \cdot \Pr(X = x).$$

Example: Roll a fair die. $X =$ value shown.

$$\mathbb{E}[X] = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = 3.5$$

Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (even if dependent!):

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

For a constant c: $\mathbb{E}[cX] = c\,\mathbb{E}[X]$ and $\mathbb{E}[X + c] = \mathbb{E}[X] + c$.

Linearity of Expectation — Example

Example: Expected number of heads in n coin flips.

- Let $X_i = 1$ if flip i is heads, 0 otherwise.
- $\mathbb{E}[X_i] = p$ for each i.
- $\mathbb{E}[\sum_{i=1}^{n} X_i] = np$.

Linearity of Expectation — Puzzle

Puzzle: n students in a class. Each student randomly picks a name from a hat (with replacement). What is the expected number of students who pick their own name?

- Let $X_i = 1$ if student i picks their own name, 0 otherwise.
- $\mathbb{E}[X_i] = 1/n$ for each i.
- $\mathbb{E}[\sum_{i=1}^{n} X_i] = n \cdot (1/n) = 1$.

Markov's Inequality

If $X \geq 0$, then for any $a > 0$:

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

Markov's Inequality

If $X \geq 0$, then for any $a > 0$:

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

Example: Average grade is 70. What is the probability that a random student scores ≥ 90?

$$\Pr(X \geq 90) \leq \frac{70}{90} = \frac{7}{9} \approx 0.78$$

Bloom Filters

The Membership Problem

Goal: Given a set S, answer queries "is $x \in S$?"

Motivating example: S is the set of malicious URLs.

Exact solution: hash table.

- Uses $\Theta(|S| \times \text{item size})$ words of space.
- A URL of 200 characters = 200 bytes.
- For $|S| = 10^7$: $\approx$ **1.8 GiB** of memory.

Can We Do Better?

Idea: Use $O(|S|)$ *bits* instead of words.

- For $|S| = 10^7$: only **1.5 MiB** of memory.
- $1000\times$ smaller than the hash table solution.

Applications: spell checkers, network routers, database joins, web caches.

The Trade-off

A Bloom filter uses far less space by allowing **false positives** and not having to store the actual elements.

- **False positive**: filter says "yes" but $x \notin S$.
- **False negative**: *never happens*. If $x \in S$, filter always says "yes".

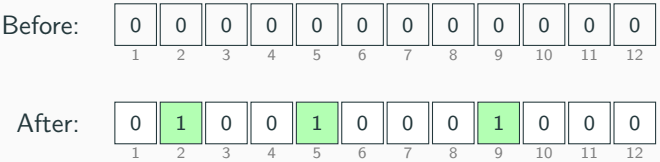
We control the false positive rate ε by choosing filter parameters.

Data Structure

- $|S| \leq n$
- m-bit array $B[1 \ldots m]$, initially all $\mathbf{0}$.
- k independent hash functions $h_1, \ldots, h_k : \mathcal{U} \to \{1, \ldots, m\}$.
- **Insert**(x): For $i = 1$ to k: set $B[h_i(x)] \leftarrow 1$.
- **Query**(x): Return **yes** iff $B[h_i(x)] = 1$ for all i.

Visualization: Insert x

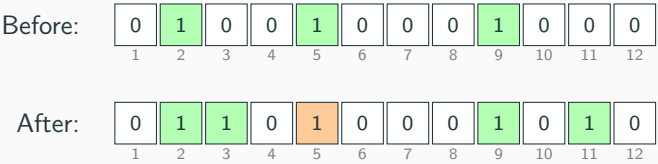
$k = 3$, $m = 12$. Insert x with $h_1(x) = 2$, $h_2(x) = 5$, $h_3(x) = 9$.



Bits 2, 5, 9 are set to 1.

Visualization: Insert y

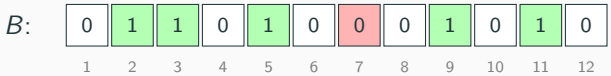
Insert y with $h_1(y) = 3,\ h_2(y) = 5,\ h_3(y) = 11$.



Bits 3 and 11 are newly set. Bit 5 was already 1 (**collision**).

Visualization: True Negative

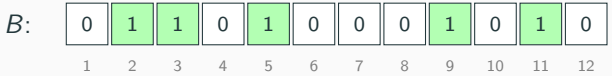
Query z with $h_1(z) = 3$, $h_2(z) = 7$, $h_3(z) = 11$:



- Check positions 3, 7, 11: $B[3] = 1$, $B[7] = 0$, $B[11] = 1$.
- $B[7] = 0 \Rightarrow$ answer is **no**. Correct: z was never inserted.

Visualization: False Positive

Query z' with $h_1(z') = 2$, $h_2(z') = 5$, $h_3(z') = 9$:



- Check positions 2, 5, 9: all equal 1 $\Rightarrow$ answer is **yes**.
- But z' was *never inserted* — this is a **false positive**.
- The bits were set by x, not by z'.

Probability a Bit is Zero

After inserting n elements using k hash functions into m bits:

Each insertion sets a uniformly random bit in each row. Use the approximation $1 - x \approx e^{-x}$ for small x. The probability a specific bit $B[j]$ is **never set**:

$$\Pr[B[j] = 0] = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

More insertions $\Rightarrow$ fewer zero bits $\Rightarrow$ more false positives.

False Positive Probability

A false positive occurs when *all* k probed bits equal 1.

Each probed bit is 1 with probability $1 - e^{-kn/m}$, independently:

$$p_{\text{fp}} \approx \left(1 - e^{-kn/m}\right)^k$$

Choosing Optimal Parameters

Set $k = \log_2(1/\varepsilon)$ and $m = n \log_2(1/\varepsilon)/\ln 2$:

$$p_{\text{fp}} \approx \left(1 - e^{-\ln 2}\right)^{\log_2(1/\varepsilon)} = \left(\frac{1}{2}\right)^{\log_2(1/\varepsilon)} = \varepsilon$$

We can **tune** ε **directly** by choosing k and m accordingly.

Space Requirement

- Required: $m = O\left(n \log_2 \frac{1}{\varepsilon}\right)$ bits.

optimal k^*	false positive rate
3	$\approx 9.2\%$
6	$\approx 2.2\%$
7	$\approx 0.82\%$
11	$\approx 0.046\%$

Doubling m/n roughly **squares** the false positive rate.

Space and Time

- **Space**: $m = O(n \log(1/\varepsilon))$ bits.
- **Insert**: $O(k)$ hash evaluations.
- **Query**: $O(k)$ hash evaluations.
- With $k = \log_2(1/\varepsilon)$: both operations take $O(\log(1/\varepsilon))$ time.